

Multi-Language Vulnerability Detection: Approaches, Challenges, and Opportunities

Darryl Max (23477658)

April 27, 2026

1 Introduction

Software vulnerabilities are flaws in computer programs that can be used to gain unauthorized access, execute code remotely, or compromise system functionality. The scale of this problem has grown steadily over the years in relation to the world becoming more digital, with reported vulnerabilities increasing from 321 Common Vulnerabilities and Exposures (CVE) records in 1999 to over 29,000 in 2023[26], demonstrating a need for effective detection methodologies. Vulnerabilities typically arise from programming errors, design flaws, or improper implementation of security controls during development[24], creating entry points for cyberattacks that can result in significant financial losses and system compromises across interconnected digital infrastructures.

The evolution of vulnerability detection has progressed through several distinct phases, transforming from basic manual security practices in early computing to today’s sophisticated AI-driven systems. The early computing era of the 1980s-1990s relied primarily on manual code review and security through obscurity until pivotal events like the 1988 Morris Worm highlighted the need for systematic approaches[18]. This led to the standardization period of the late 1990s with the introduction of the CVE system, followed by the automation era of the early 2000s that brought structured vulnerability scanners and patch management systems. The current proactive systems era has shifted from reactive to predictive approaches, leveraging machine learning and deep learning techniques to identify threats before they can be exploited[18].

This literature review examines the feasibility of unified vulnerability detection models that can operate across multiple programming languages, addressing the limitations of a single-language research focus and the multilingual requirements of contemporary software systems. By analyzing existing approaches and identifying cross-language vulnerability patterns, this review establishes the theoretical foundation for developing models that can provide comprehensive security analysis across C/C++, Python, Java, and JavaScript codebases within unified frameworks.

1.1 Overview

This literature review is structured to progressively establish the case for unified multi-language vulnerability detection. Section 2 establishes motivation by documenting polyglot development prevalence and associated security challenges. Section 3 demonstrates that fundamental vulnerability patterns transcend language syntax through concrete cross-language examples. Section 4 reviews existing transformer-based and graph-based detection approaches, examining their architectures, capabilities, and limitations. Section 5 provides systematic comparison of these paradigms, identifying complementary strengths. Section 6 addresses dataset quality and availability, identifying the critical infrastructure gap of absent multi-language benchmarks. Section 7 synthesizes findings into specific research directions for advancing unified vulnerability detection. This organization progresses from establishing necessity through demonstrating feasibility to identifying paths forward.

2 Motivation for research

2.1 Polyglot development practices in the real world

Polyglot software development—using multiple programming languages within single projects—has become the dominant paradigm in modern software engineering. Empirical evidence demonstrates that over 80% of projects employ multiple languages, with professional developers averaging seven languages per project [24, 13]. This architectural pattern has evolved from exception to standard practice, fundamentally reshaping how contemporary systems are constructed.

Four principal drivers motivate this widespread polyglot adoption. First, *technical optimization* leverages language-specific strengths for particular problems: Python excels at data analysis, JavaScript dominates client-side interaction, C++ provides performance-critical computation, and SQL manages persistence [24]. Second, *architectural separation* uses language boundaries to delineate system components—user interfaces (HTML/CSS), business logic (Java/Python), and infrastructure automation (Bash/Docker)—improving maintainability through explicit interfaces [15]. Third, *ecosystem leverage* enables access to diverse, specialized libraries across language communities rather than reimplementing functionality or accepting single-language constraints [15]. Finally, *organizational flexibility* allows teams to utilize existing developer expertise and modern language features rather than requiring wholesale retraining on legacy monolithic stacks.

Yet this architectural flexibility carries a hidden cost: the language boundaries that enable clean separation simultaneously introduce substantial security vulnerabilities, as the following section demonstrates.

2.2 Security Implications and Vulnerability Landscape

The architectural flexibility of polyglot systems introduces substantial security risks that manifest specifically at language boundaries where different execution models, type systems, and security mechanisms interact. Empirical studies reveal that over 90% of developers encounter problems related to language interactions, with 38% reporting interfacing issues and 30% experiencing data handling complications [24]. More critically, polyglot systems demonstrate significantly increased vulnerability to both functional defects and security flaws compared to single-language implementations, as inconsistencies in how languages handle data, memory, and security propagate across component boundaries.

2.2.1 Fundamental Challenges

Three fundamental incompatibilities drive vulnerability emergence in polyglot systems. *Memory management conflicts* arise when languages with manual memory control (C/C++) interface with garbage-collected languages (Java, Python), creating conditions for buffer overflows, use-after-free errors, and memory leaks that would not occur within homogeneous systems [24]. *Type system mismatches* across languages with varying typing disciplines—static versus dynamic, strong versus weak—make data conversions error-prone, particularly when third-party libraries provide different implementations of ostensibly identical algorithms with incompatible parameters [15]. Finally, *security boundary inconsistencies* emerge when authentication, authorization, or input validation implemented in one language component fails to propagate correctly across language interfaces, enabling bypasses and injection attacks at transition points.

These architectural challenges are compounded by practical development obstacles: insufficient documentation for cross-language APIs, uninformative compiler errors at language boundaries, inadequate tooling for multi-language builds, and particularly critical for security, the difficulty of testing language interactions where test oracles must integrate program states across different execution models [15].

2.2.2 Vulnerability Manifestations

The fundamental incompatibilities described above manifest as specific, exploitable vulnerability classes that are particularly prevalent in polyglot contexts:

Injection Vulnerabilities emerge at data flow boundaries between languages. Cross-Site Scripting (XSS) attacks exploit inadequate sanitization when data passes from server-side languages to client-side JavaScript, injecting malicious scripts into trusted web pages [17]. SQL Injection vulnerabilities arise when improper input validation in application languages allows attacker-controlled data to alter database queries, a risk amplified in polyglot systems where multiple language layers interface with persistence mechanisms [17].

Authorization and Authentication Bypass vulnerabilities exploit security boundary inconsistencies, where access control checks implemented in one language are circumvented when requests route through different language components. The lack of unified security enforcement across language boundaries creates gaps in session management, permission verification, and identity validation [17].

Memory Corruption Vulnerabilities—including buffer overflows, use-after-free, and null pointer dereferences—reflect the memory management conflicts inherent to polyglot systems. These flaws are especially problematic when manually-managed C/C++ components interact with garbage-collected languages, as assumptions about object lifetime and memory ownership differ fundamentally between memory models [17].

While this review focuses on these three vulnerability classes due to their prevalence at language interfaces, polyglot systems remain susceptible to additional vulnerability types including command injection, insecure deserialization, XML external entity (XXE) attacks, cross-site request forgery (CSRF), type confusion, and remote code execution. The selection of these primary classes reflects their documented frequency in multi-language contexts and their direct connection to the fundamental incompatibilities—memory management, type system mismatches, and security boundary inconsistencies—that characterize polyglot architectures. Critically, these vulnerabilities often manifest not within individual language components, where established detection tools perform adequately, but rather at the interfaces where languages interact—precisely where current detection approaches exhibit significant blind spots.

This detection gap is exacerbated by tool fragmentation: developers working on polyglot systems must employ separate language-specific tools for each programming language, leading to inconsistent coverage, incompatible reporting formats, and substantial maintenance overhead. Given that developers average seven languages per project [24], this fragmented landscape represents a barrier to comprehensive security analysis. Yet as the following section demonstrates, the vulnerability patterns themselves may provide a path forward.

3 Towards Multi-Language Vulnerability Detection

3.1 Vulnerability Patterns Transcend Language Syntax

The feasibility of such a unified approach is supported by a key observation: for example, common vulnerability classes such as SQL injection and authorization bypass exhibit recognizable patterns regardless of the specific language syntax in which they are expressed.

3.1.1 SQL Injection Patterns

SQL injection fundamentally involves unsanitized user input being concatenated into database queries. This pattern manifests across languages:

Listing 1: Python SQL Injection

```
1 # Python
2 query = "SELECT * FROM users WHERE id = " + user_input
3 cursor.execute(query)
```

Listing 2: JavaScript SQL Injection

```

1 // JavaScript/Node.js
2 const query = 'SELECT * FROM users WHERE id = ${userId}';
3 db.query(query);

```

Listing 3: C# SQL Injection

```

1 // C#
2 string query = "SELECT * FROM users WHERE id = " + Request.QueryString["id"];
3 command.ExecuteReader(query);

```

The syntax varies significantly across these languages—different concatenation operators (+ vs . vs #{}), distinct string interpolation styles (\${ } vs #{}), varied parameter access methods (request.getParameter vs \$.GET vs params). Yet the vulnerability pattern is identical: untrusted data flows directly into a SQL query without parameterization or sanitization. The core pattern can be abstracted as: **Untrusted data** → **String concatenation/interpolation** → **Database query execution**.

3.1.2 Authorization Bypass Patterns

Authorization vulnerabilities manifest as missing or inconsistent access control checks before sensitive operations:

Listing 4: Python Authorization Bypass

```

1 # Python/Flask
2 @app.route('/admin/delete_user/<user_id>')
3 def delete_user(user_id):
4     # Missing authorization check
5     User.query.filter_by(id=user_id).delete()
6     return "User deleted"

```

Listing 5: Java Authorization Bypass

```

1 // Java/Spring
2 @DeleteMapping("/admin/users/{userId}")
3 public ResponseEntity<String> deleteUser(@PathVariable String userId) {
4     // Missing authorization check
5     userRepository.deleteById(userId);
6     return ResponseEntity.ok("User deleted");
7 }

```

Listing 6: C# Authorization Bypass

```

1 // C#/ASP.NET
2 [HttpDelete("admin/users/{userId}")]
3 public IActionResult DeleteUser(string userId)
4 {
5     // Missing authorization check
6     _userRepository.Delete(userId);
7     return Ok("User deleted");
8 }

```

Despite different web frameworks (Flask, Express, Spring, Rails, ASP.NET), routing mechanisms, and method signatures, the pattern is consistent: sensitive operations execute without verifying the caller has appropriate permissions. The abstracted pattern is: **Sensitive operation** → **Missing permission check** → **Execution**.

3.1.3 Memory Corruption Patterns

Memory corruption vulnerabilities arise from unsafe buffer or array operations without proper bounds validation. While the manifestation differs across memory management models, the fundamental pattern remains consistent:

Listing 7: C/C++ Buffer Overflow

```
1 // C/C++
2 void process_input(char *user_input) {
3     char buffer[64];
4     strcpy(buffer, user_input); // No bounds checking
5     process(buffer);
6 }
```

Listing 8: Java Array Index Vulnerability

```
1 // Java
2 public void processData(int userIndex, String[] data) {
3     // Missing bounds validation
4     String value = data[userIndex];
5     processValue(value);
6 }
```

Listing 9: Python Buffer Operation

```
1 # Python with ctypes (native interop)
2 buffer = ctypes.create_string_buffer(64)
3 user_data = get_user_input()
4 # No length validation before copying
5 ctypes.memmove(buffer, user_data, len(user_data))
```

While the severity differs across languages—C/C++ buffer overflows enable arbitrary code execution, Java throws runtime exceptions, and Python’s memory safety depends on native extension behavior—the underlying pattern is identical: **Untrusted size/index** → **Buffer/array operation** → **Missing bounds validation**. In polyglot systems, this pattern becomes particularly dangerous when data crosses language boundaries: a size parameter validated in Java may overflow when passed to a C extension, or a Python string length may exceed C buffer allocations, exploiting the mismatch between garbage-collected and manually-managed memory models.

Despite dramatic syntactic differences, these examples reveal fundamental pattern consistency: SQL injection involves untrusted data in query construction, authorization bypass reflects missing permission checks, memory corruption stems from inadequate bounds validation. This consistency suggests vulnerability detection need not be bound to language-specific syntax. A model that recognizes abstract vulnerability structures—untrusted data flows, absent validation, unsafe operations—could potentially operate across multiple languages within unified frameworks. The critical question becomes whether existing detection approaches can effectively learn and generalize these patterns across syntactic boundaries, which the following section examines.

4 AI Approaches to Vulnerability Detection

Given the prevalence of polyglot development and the demonstrated consistency of vulnerability patterns across languages, the question becomes: what automated approaches can effectively detect these cross-language patterns? Two deep learning paradigms have emerged in recent vulnerability detection research: transformer-based models, particularly BERT and its variants, and graph-based approaches using Graph Neural Networks (GNNs). While both have shown promise for single-language detection, their capabilities for the multi-language detection required in polyglot systems remain largely unexplored. Below is the breakdown of the approaches.

4.1 BERT: Bidirectional Encoder Representations from Transformers

4.1.1 Architecture

BERT is fundamentally a stack of Transformer encoder layers, each consisting of multiple self-attention "heads" [16]. For every input token in a sequence, each attention head forms key, value, and query vectors to determine token-to-token relationships through learned contextual weighting.. The outputs from all heads in a layer are combined through a fully connected layer, with skip connections and layer normalization applied at each stage.

The key architectural innovation of BERT is its **bidirectional training approach**. Unlike previous models that processed text left-to-right or right-to-left, BERT considers context from both directions simultaneously [5]. This bidirectional understanding enables BERT to capture deeper semantic and syntactic relationships within text—or in the case of vulnerability detection, within source code.

4.1.2 Pre-training, Fine-tuning, and Learned Representations

BERT follows a two-stage workflow particularly relevant to vulnerability detection:

- *Masked Language Modelling (MLM)* - During **pre-training**, BERT is trained on massive amounts of unlabeled text using two self-supervised objectives: *Masked Language Modeling (MLM)*, where random tokens are masked and the model learns to predict them based on surrounding context [16]
- *Next Sentence Prediction (NSP)*- where the model learns whether two sentences naturally follow each other. During **fine-tuning**, the pre-trained model is adapted to specific downstream tasks (such as vulnerability classification) by adding task-specific layers and training on labeled data. For vulnerability detection specifically, the pre-trained BERT model can be fine-tuned on datasets of vulnerable and non-vulnerable code snippets, enabling it to learn patterns indicative of security flaws. [16]

During **fine-tuning**, the pre-trained model is adapted to specific downstream tasks by adding a task-specific classification layer and training on labeled data for that task. For vulnerability detection, this means taking a BERT model pre-trained on general code (or text) and fine-tuning it on datasets of labeled vulnerable and non-vulnerable code snippets. The model's pre-trained representations provide a strong initialization, allowing effective learning even with limited labeled vulnerability data—a critical advantage given that security-labeled datasets are expensive and time-consuming to construct [8].

This two-stage paradigm enables transfer learning: general patterns learned during pre-training on massive unlabeled corpora transfer to specialized tasks through fine-tuning, requiring far less task-specific labeled data than training from scratch. For code analysis across multiple programming languages, this means a model can be pre-trained on large amounts of unlabeled code from diverse languages, then fine-tuned for vulnerability detection, potentially leveraging cross-language patterns learned during the pre-training phase.

4.1.3 Code-Specialized BERT Variants

While vanilla BERT was designed for natural language, its success has led to specialized variants optimized for programming languages and code understanding tasks. These code-specific models consistently outperform vanilla BERT on code-related tasks, demonstrating that code-specific pre-training is essential for capturing the unique characteristics of programming languages.

CodeBERT [8] is a pre-trained transformer model specifically designed to understand programming languages by learning from both code and its natural language documentation simultaneously through bimodal pre-training. Unlike vanilla BERT, which treats code purely as text, CodeBERT is trained to understand the semantic relationship between what code does (expressed in natural language) and how it does it (expressed in programming syntax). Most significantly for multi-language vulnerability

detection, CodeBERT has been trained on six programming languages (Python, Java, JavaScript) simultaneously without language-specific markers, enabling it to learn language-agnostic programming concepts and patterns [8]. The model’s successful zero-shot transfer to C#—a language entirely absent from training data—demonstrates it captures fundamental programming abstractions rather than memorizing language-specific syntax.

GraphCodeBERT [10] extends CodeBERT by incorporating program structure through data flow graphs alongside token sequences. This integration enables better understanding of code semantics beyond sequential token patterns, capturing variable dependencies and data flow relationships explicitly. The model jointly learns from source code tokens, abstract syntax trees, and data flow edges, providing richer code representations that understand both what code says (tokens) and what it means (structure).

CodeT5 [22] employs a unified encoder-decoder architecture that leverages code-specific knowledge including identifier-aware pre-training, which preserves the semantic meaning of variable and function names, and bimodal dual generation, enabling the model to both understand and generate code. This architecture makes CodeT5 particularly effective for tasks requiring both code comprehension and synthesis.

UniXcoder [9] is a unified cross-modal pre-trained model that supports both code understanding and generation across multiple programming languages. It employs cross-modal contrastive learning between code and comments, along with masked language modeling and span corruption objectives, to learn unified representations applicable to diverse downstream tasks including code search, translation, and summarization.

These specialized variants demonstrate that while vanilla BERT provides a strong foundation, code-specific architectural enhancements and pre-training objectives substantially improve performance on programming language tasks. However, their application to vulnerability detection across multiple languages remains largely unexplored, representing a significant research opportunity.

4.1.4 Advantages for Multi-Language Vulnerability Detection

BERT’s architecture offers several compelling advantages for unified vulnerability detection across polyglot systems. The model captures both syntactic structure (hierarchical code organization, dependency relationships) and semantic meaning (what code constructs represent) through its learned representations [16, 11, 19]. Most critically, BERT produces contextualized representations where identical tokens receive different encodings based on surrounding context [6], enabling it to recognize that syntactically different expressions across languages may represent semantically equivalent vulnerability patterns—for instance, identifying that `query = "SELECT..." + user_input` in Python and `String query = "SELECT..." + userId` in Java both represent SQL injection despite syntactic differences. The transfer learning paradigm allows models pre-trained on large multilingual code corpora to adapt to vulnerability detection with limited labeled data, potentially enabling patterns learned from data-rich languages to transfer to languages with sparse security datasets [8]. Code-specialized variants, particularly CodeBERT, demonstrate empirically validated multi-language capability through successful zero-shot transfer to unseen languages and consistent cross-language performance [8], distinguishing transformer approaches from methods with merely theoretical cross-language potential. Finally, bidirectional attention captures long-range dependencies essential for detecting vulnerabilities spanning distant code elements, while end-to-end learning eliminates the need for language-specific preprocessing infrastructure, enabling a single model to analyze polyglot codebases without separate parsing tools for each language.

4.1.5 Limitations and Challenges

Despite these advantages, transformer-based approaches face significant constraints for vulnerability detection. Because BERT processes up to 512 tokens at once, it may miss important context in longer code segments [16]. Research demonstrates that BERT shows surprising insensitivity to malformed input and may rely on shallow heuristics rather than deep structural comprehension [7, 16], limiting effectiveness for vulnerabilities requiring precise control-flow or data-flow analysis. Explainability

analyses reveal sensitivity to superficial token patterns—such as function names containing “vulnerable”—rather than genuine security indicators [16], raising concerns about whether models learn meaningful vulnerability patterns or memorize dataset artifacts. The model also exhibits substantial overparameterization with redundant attention heads [14, 21], impacting computational efficiency without corresponding accuracy benefits. These limitations suggest that while transformer-based models provide a strong foundation for multi-language vulnerability detection, particularly through CodeBERT’s validated cross-language capabilities, they may benefit from augmentation with explicit structural information or hybrid approaches for vulnerabilities requiring precise program analysis.

4.2 Graph Neural Networks

Another approach in deep learning has led researchers to explore Graph Neural Networks (GNNs) as a promising approach for automated vulnerability detection. Unlike traditional static analysis or pattern-matching techniques, GNNs can learn complex structural patterns from code representations, potentially capturing subtle vulnerabilities that require understanding both syntactic and semantic properties of programs.

4.2.1 GNN Architectures for Vulnerability Detection

The application of GNNs to vulnerability detection has converged on a common architectural pattern, though specific implementations vary across approaches. The general architecture consists of an initial graph embedding phase, followed by multiple GNN layers for propagating and aggregating information across the graph structure, then feature extraction modules that consolidate learned representations, and finally classification layers that produce vulnerability predictions. Within this framework, researchers have explored different variants of graph neural networks. Some approaches leverage Graph Attention Networks (GAT) with attention mechanisms to weight the importance of different code relationships, as demonstrated in LineVD’s two-layer GAT [12] architecture combined with convolutional layers for classification. Others employ Gated Graph Neural Networks (GGNN), with innovations such as bidirectional message passing where backward edges complement traditional forward edges to enable richer information flow, as implemented in BGNN4VD[1]. Alternative architectures incorporate Gated Graph Recurrent Layers with specialized convolutional modules designed for hierarchical feature extraction from code graphs, as seen in Devign’s approach[26]. Despite these architectural variations, the unifying principle remains the transformation of code into graph structures that enable neural networks to learn vulnerability patterns through iterative neighborhood aggregation and feature refinement.

4.2.2 Graph Representations of Code

A critical insight underlying GNN-based vulnerability detection is that effective analysis requires composite graph structures that integrate multiple complementary views of source code. Contemporary approaches construct unified graphs that merge information from several traditional program analysis abstractions

- Abstract Syntax Tree (AST): Provides the foundational node structure, capturing the syntactic organization of code elements and their hierarchical relationships. AST nodes typically represent program constructs such as statements, expressions, and declarations [26, 1, 12].
- Control Flow Graph (CFG): Contributes edges representing possible execution paths through the program, enabling the model to understand how control transfers between different code segments. These edges capture branching, looping, and sequential execution patterns[26, 1, 12].
- Data Flow Graph (DFG): Adds edges that track how variables are defined, used, and modified throughout execution. This representation is essential for detecting vulnerabilities related to improper data handling, such as use-after-free or uninitialized variable access[26, 1, 12].

- Program Dependence Graph (PDG): Explicitly combines both control and data dependencies into a unified representation, capturing which statements depend on which other statements for both their execution and their data values[12].
- Natural Code Sequence (NCS): Preserves the sequential ordering of code tokens alongside structural information, maintaining the linear flow of the original source code within the graph representation[26].

By merging these diverse perspectives into a single graph where nodes typically represent AST elements and edges encode multiple types of relationships simultaneously, GNNs can access the rich, multi-faceted semantics necessary for identifying complex vulnerabilities.

4.2.3 Learning from Code Structure

The power of GNNs for vulnerability detection stems from their ability to learn hierarchical patterns at multiple levels of abstraction from code structure. At the local level, GNNs capture fine-grained patterns such as variable dependencies within individual statements and control flow within basic code blocks [12]. Through successive layers of message passing and aggregation, these local patterns are progressively combined to form global, function-wide representations that characterize vulnerability patterns across entire code units[25]. This hierarchical learning is particularly valuable because, as researchers have observed, the majority of vulnerabilities such as memory leaks are too subtle to be spotted without a joint consideration of composite code semantics. A memory leak, for instance, cannot be reliably detected by examining only the syntactic structure or only the control flow; rather, it requires understanding both where memory is allocated and deallocated in the control flow and how pointers to that memory are used and modified in the data flow [26]. GNNs excel at learning these composite patterns because their iterative neighborhood aggregation naturally integrates information from multiple edge types simultaneously, allowing the model to discover vulnerability signatures that depend on complex interactions between syntax, control flow, and data flow properties [26, 1].

4.2.4 Potential for Multi-Language Detection

A frequently cited advantage of GNN-based approaches is their potential generalizability across programming languages. Proponents argue that because the underlying graph representations such as ASTs, CFGs, and DFGs are conceptual abstractions independent of specific programming language syntax, techniques built upon these representations are not inherently tied to any particular language [1, 26]. This language-agnostic foundation suggests that a GNN trained on one language might transfer to others with minimal adaptation, or that a single model could potentially learn vulnerability patterns applicable across multiple languages simultaneously. However, the empirical evidence for this claim remains limited. Existing evaluations have been restricted almost exclusively to C and C++ codebases [12, 1, 26], leaving the cross-language generalization capabilities largely unexplored. Furthermore, while the graph structures themselves may be language-independent in principle, practical implementations often rely on language-specific tokenization and embedding techniques, such as word2vec models trained on language-specific corpuses, which would require retraining or adaptation for different languages [26]. Thus, while the theoretical foundation for multi-language detection is compelling, substantial empirical validation is needed to determine whether GNN-based approaches can truly achieve practical cross-language vulnerability detection.

4.2.5 Limitations and Challenges

Despite their promise, GNN-based vulnerability detection faces significant limitations constraining practical deployment. Performance remains inadequate, with F1 scores ranging from 36% [12] to 73% [26], and precision on real CVEs around 45% [1]. These systems particularly struggle with irregular code patterns like goto statements and complex control structures [12]. Next, scalability poses major challenges: function size limits (400-500 nodes) exclude larger, complex functions [1, 26]; the "neighbor explosion" problem causes exponential growth in computation with network depth

[25]; and over-smoothing in deeper networks reduces distinguishability between vulnerable and benign patterns [25]. Technical constraints further limit these approaches. Dependency on preprocessing tools like Joern introduces failure points [26], while restriction to intra-procedural analysis misses inter-procedural vulnerabilities [12]. Runtime properties and design-level flaws remain entirely out of scope [26]. Data challenges compound these issues: creating labeled datasets requires extensive manual effort (600 person-hours reported) [26]; severe class imbalance complicates learning [12, 1]; and reliance on synthetic vulnerabilities may not capture real-world complexity [26]. These accumulated challenges indicate that while GNNs offer promising capabilities for learning from rich code representations, significant advances in reliability, scalability, and scope are necessary before practical adoption in production security workflows.

5 Comparative Analysis: Transformer-Based vs. Graph-Based Approaches

Having examined both transformer-based models and graph neural network approaches for vulnerability detection, this section provides a systematic comparison of these two dominant paradigms to understand their relative strengths, limitations, and suitability for multi-language vulnerability detection in polyglot systems.

The fundamental distinction between these approaches lies in how they represent and process source code. Transformer-based models like BERT and CodeBERT treat code as sequences of tokens, processing them through self-attention mechanisms that capture relationships between all token pairs [5, 8]. This sequential representation, enhanced by positional embeddings, fundamentally views code as text-like data where meaning emerges from token order and contextual windows. In contrast, GNN-based approaches construct explicit graph structures—typically composite graphs merging Abstract Syntax Trees (AST), Control Flow Graphs (CFG), Data Flow Graphs (DFG), and Program Dependence Graphs (PDG)—that explicitly encode structural relationships governing program execution [26, 1]. These representations are not learned but extracted through static analysis tools like Joern. This representational difference has profound implications: BERT must learn implicit structural relationships from token sequences, while GNNs operate on explicit structural representations but must learn which structural patterns indicate vulnerabilities.

The learning paradigms differ substantially between the two approaches. Transformer-based models leverage massive unsupervised pre-training on unlabeled code corpora followed by task-specific fine-tuning, enabling effective learning even with limited labeled vulnerability data—a critical advantage given the expense of creating security-labeled datasets [8]. GNN-based approaches typically require supervised training with function-level vulnerability labels from the outset, limiting their applicability when labeled data is scarce [26]. This difference in data efficiency significantly impacts practical deployment, particularly for languages or vulnerability types where labeled examples are rare.

Regarding structural knowledge, GNNs encode program structure explicitly through extracted graphs (AST, CFG, DFG), providing precise access to control-flow and data-dependency relationships essential for detecting vulnerabilities like memory leaks that depend on exact program execution paths [26]. Transformers must learn these structural relationships implicitly from token sequences, resulting in incomplete and sometimes unreliable structural understanding that shows surprising insensitivity to malformed input [16]. However, this implicit learning enables transformers to discover relevant patterns without preprocessing dependencies that can fail, offering end-to-end learning advantages [8].

The most critical difference for polyglot vulnerability detection concerns multi-language capability. CodeBERT demonstrates empirically validated multi-language capabilities through successful performance on six programming languages and zero-shot transfer to C# [8], providing concrete evidence that transformer models can learn language-agnostic programming patterns. In contrast, while GNN approaches remain theoretically language-agnostic due to their use of abstract graph representations independent of specific syntax, existing evaluations have been restricted almost exclusively to C and C++ [26, 1, 12]. The practical multi-language applicability of GNNs thus remains an open empirical question, as implementations often rely on language-specific tokenization and word2vec embeddings that would require retraining for different languages.

Context modeling approaches also differ significantly. BERT’s self-attention mechanism enables modeling of long-range dependencies within its 512-token context window, with each token attending to all others [20]. However, this window constraint can truncate larger functions or exclude critical context [16]. GNNs model context through iterative neighborhood aggregation where each node’s representation incorporates information from neighbors in the graph, naturally following program structure along control-flow and data-flow edges. Yet GNNs face the “neighbor explosion” problem where deep message-passing becomes computationally prohibitive as neighborhoods grow exponentially, alongside over-smoothing issues that reduce distinguishability in deeper networks [26].

Performance metrics reveal substantial differences in maturity and effectiveness. Current GNN-based vulnerability detection achieves F1 scores ranging from 36% to 73% with precision around 45% on real CVEs [12, 26, 1]—insufficient for practical deployment where false positive rates directly impact developer trust and adoption. CodeBERT achieves notably higher performance on code understanding tasks (76% MRR) [8], though direct comparison is challenging due to different evaluation contexts and the lack of standardized multi-language vulnerability detection benchmarks.

From a practical deployment perspective, transformer-based models offer end-to-end learning with minimal preprocessing but require substantial computational resources for models with 125M+ parameters [8]. GNN-based models depend on reliable static analysis tools for graph extraction, are restricted by function size constraints (typically 400-500 nodes), but may offer more interpretable predictions through explicit graph structures that show exactly which code paths contributed to vulnerability predictions [26, 1].

For the specific goal of unified multi-language vulnerability detection in polyglot systems, this comparison reveals that transformer-based models, particularly CodeBERT, currently provide the strongest foundation due to empirically validated cross-language capabilities and superior performance. The model’s successful zero-shot transfer to C# and consistent performance across six training languages demonstrates genuine language-agnostic pattern learning rather than syntax memorization. However, transformers lack the structural precision that GNNs provide for detecting control-flow and data-flow dependent vulnerabilities, suggesting potential benefits from hybrid approaches that combine transformer semantic understanding with graph-based structural reasoning. Critically, neither approach explicitly models language boundaries where polyglot vulnerabilities emerge through memory management conflicts and type conversion issues, nor do standardized multi-language vulnerability detection benchmarks currently exist to rigorously evaluate cross-language generalization. These gaps represent significant opportunities for future research in developing unified detection frameworks that match the multi-language reality of modern software development.

6 Datasets

6.1 Dataset Quality and Evaluation Metrics

Effective vulnerability detection research fundamentally depends on high-quality datasets and appropriate evaluation metrics. Several critical characteristics distinguish valuable datasets from inadequate ones, directly impacting model performance, generalizability, and practical applicability.

Real-World Representativeness: Research consistently demonstrates that models trained on real-world code from production systems significantly outperform those trained on synthetic test cases [23]. Real-world code exhibits substantially greater complexity—production code averages 176 lines with 8 branches compared to synthetic datasets averaging 20 lines with 1 branch—reflecting the genuine patterns, coding styles, and architectural decisions of actual software development [23]. Synthetic datasets, while offering advantages such as balanced classes and controlled complexity, demonstrate limited real-world applicability due to simplified patterns that fail to capture the diversity and subtlety of production vulnerabilities.

As illustrated in Figure 1, the distribution differences between real-world and synthetic datasets are substantial. GitHub samples exhibit significantly greater structural complexity with longer functions (up to 500 lines versus 120) and more control-flow branches (up to 20 versus 4), reflecting the genuine complexity of production code. This complexity difference directly impacts model effectiveness: detec-

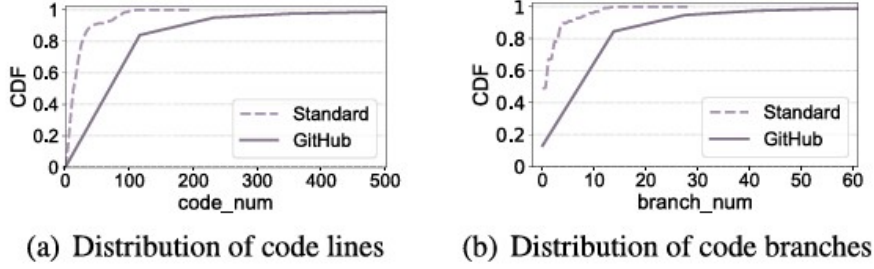


Figure 1: Distribution of code lines and control-flow branches in vulnerability samples from GitHub (real-world) versus Standard (SARD) datasets. The cumulative distribution functions reveal that GitHub samples exhibit substantially greater complexity: code lines distributed across 0-500 range versus 0-120 for Standard, and branches distributed across 0-20 versus 0-4. This complexity difference reflects genuine production code patterns essential for training robust detection models. Adapted from Xu et al. [23].

tion models trained on GitHub’s real-world data achieve F1-scores of 84.1 compared to only 59.2 when trained on Standard’s synthetic data, demonstrating that dataset quality fundamentally influences detection capabilities [23].

Scale and Diversity: Modern datasets should contain tens to hundreds of thousands of labeled samples to enable robust model training. Equally important is diversity across multiple dimensions: samples from numerous projects (rather than a single codebase) ensure models learn general vulnerability patterns rather than project-specific artifacts; coverage of diverse CWE categories (ideally dozens to over a hundred) enables detection of varied vulnerability types; and representation of different coding styles, frameworks, and architectural patterns improves generalization to unseen code.

Label Quality and Granularity: High-quality labels require validation through CVE databases, security expert review, or confirmed vulnerability fixes. Function-level granularity represents the current standard, providing sufficient context for detection while remaining actionable for developers. However, line-level annotations—identifying exactly which lines are vulnerable—would enable more precise localization but require substantially more labeling effort.

Class Balance Considerations: Vulnerability datasets inherently exhibit severe class imbalance, with non-vulnerable code outnumbering vulnerable code by ratios of 10:1 to 20:1 or higher, reflecting the reality that vulnerabilities are relatively rare in production systems [2, 1]. While this imbalance creates training challenges—models may achieve high accuracy by predominantly predicting code as non-vulnerable—it must be carefully managed through sampling strategies rather than artificial balancing that misrepresents real-world distributions.

Metadata Richness: Valuable datasets include comprehensive metadata providing context for each instance: CVE identifiers linking to official vulnerability reports, CWE classifications enabling analysis by vulnerability type, commit information showing exactly what changed to fix the vulnerability, severity scores indicating exploitability and impact, and textual descriptions explaining the security flaw [3, 26].

Evaluation Metrics: Vulnerability detection research typically employs standard classification metrics including precision (proportion of predicted vulnerabilities that are genuine), recall (proportion of actual vulnerabilities detected), F1-score (harmonic mean balancing precision and recall), and accuracy (overall correctness). However, given severe class imbalance, F1-score and precision-recall curves provide more meaningful evaluation than raw accuracy. For practical deployment, false positive rate is particularly critical—high false positive rates lead to developer fatigue and tool abandonment, making precision as important as recall despite the severity of missing actual vulnerabilities.

6.2 Available Vulnerability Detection Datasets

The current landscape of publicly available vulnerability detection datasets exhibits a critical limitation: overwhelming concentration on C/C++ with virtually no standardized multi-language benchmarks.

Predominantly C/C++ Datasets: The vast majority of vulnerability detection research relies on C/C++-focused datasets due to the prevalence of memory-related vulnerabilities and decades of documented CVEs. Representative datasets employed in recent research include *Devign* [26], containing approximately 27,000 functions from four large open-source projects (QEMU, FFmpeg, Linux Kernel, Wireshark) covering multiple CWE types, created through manual inspection of vulnerability-fixing commits; *Big-Vul*, with over 265,000 functions extracted from C/C++ projects with CVE identifiers; and *SARD* (Software Assurance Reference Dataset), providing approximately 65,000 code snippets, though with simpler synthetic characteristics compared to production code [23]. While these datasets enabled substantial advances in GNN and transformer-based vulnerability detection research, their single-language focus fundamentally limits evaluation of cross-language generalization capabilities.

AI-Generated Multi-Language Code: Emerging datasets evaluate vulnerabilities in AI-generated code across multiple programming languages, representing a potentially valuable resource for multi-language analysis. *DeVAIC* [4] comprises 500 code snippets generated by four AI models (Google Gemini, Microsoft Copilot, OpenAI ChatGPT, GitHub Copilot) from 125 natural language prompts across multiple programming languages. The dataset reveals that 54% of generated snippets contained at least one identifiable security vulnerability, with 35 distinct CWE types identified including cryptographic issues (CWE-327, CWE-330, CWE-347) and deserialization vulnerabilities (CWE-502) [4]. While AI-generated datasets provide multi-language coverage and reveal vulnerability patterns in machine-generated code, they serve primarily as evaluation benchmarks rather than training data, as the distribution of vulnerabilities may differ from naturally occurring patterns in human-written production code.

The Multi-Language Gap: The most critical limitation is the *complete absence of standardized multi-language vulnerability detection benchmarks with consistent labeling across programming languages*. No existing dataset provides comparable vulnerability samples across languages like Python, Java, JavaScript, and C/C++ with unified annotation standards, preventing rigorous evaluation of cross-language generalization capabilities. More critically, no datasets explicitly annotate vulnerabilities that emerge specifically at language boundaries—where memory management conflicts, type conversion issues, and security mechanism inconsistencies occur in polyglot systems. This infrastructure gap represents a fundamental barrier to advancing unified vulnerability detection approaches that match the multi-language reality of modern software development. Without such benchmarks, claims about cross-language detection capabilities remain empirically unvalidated, and meaningful comparison of approaches across languages becomes impossible.

6.3 Implications for Multi-Language Vulnerability Detection

For unified multi-language vulnerability detection, current dataset limitations represent both a significant challenge and a potential opportunity. The predominant focus on C/C++ creates a fundamental barrier to multi-language research, preventing rigorous evaluation of cross-language generalization capabilities. Function-level granularity, while appropriate for individual language analysis, may miss inter-procedural vulnerabilities spanning language boundaries in polyglot systems.

However, models like CodeBERT pre-trained on multi-language code corpora could potentially enable transfer learning from data-rich languages (C/C++, Java) to languages with sparse vulnerability datasets. This transfer learning approach leverages patterns learned from abundant security data in well-studied languages, adapting them to languages where labeled vulnerabilities are scarce. Yet empirical validation of this potential requires comprehensive multi-language benchmarks that current research lacks—representing a critical infrastructure need for advancing polyglot vulnerability detection.

7 Conclusion

This literature review has examined vulnerability detection approaches through the lens of polyglot software development, where multiple programming languages coexist within single projects—a practice now prevalent in over 80% of modern software systems [24, 13]. The analysis reveals both the challenges posed by multi-language environments and the promise of unified detection approaches.

7.1 Key Findings

Four critical insights emerge from this review:

Cross-Language Vulnerability Patterns Despite dramatic syntactic differences across programming languages, fundamental vulnerability patterns—SQL injection, authorization bypass, cross-site scripting, and memory corruption—exhibit consistent structural characteristics. This pattern consistency suggests that vulnerability detection models focused on abstract patterns rather than language-specific syntax could achieve broad applicability.

Complementary Detection Paradigms Transformer-based models (particularly CodeBERT) demonstrate validated multi-language capabilities and semantic understanding through successful zero-shot transfer across six programming languages [8]. Graph neural networks provide explicit structural reasoning but lack empirical cross-language validation beyond C/C++ [26, 1]. These complementary strengths suggest potential benefits from hybrid approaches.

Research Gaps Current approaches suffer from critical limitations: fragmented language-specific tools create practical deployment barriers; cross-language generalization remains largely unvalidated; language boundaries where polyglot vulnerabilities emerge receive insufficient attention; and standardized multi-language vulnerability detection benchmarks do not exist.

Critical Dataset Gap The overwhelming concentration of vulnerability datasets on C/C++ creates a fundamental barrier, with no standardized multi-language benchmarks existing to enable rigorous cross-language evaluation. Dataset quality directly impacts model effectiveness—models trained on real-world code substantially outperform those on synthetic data (F1: 84.1 vs. 59.2) [23]—yet comprehensive multi-language datasets with consistent labeling remain absent, representing the most critical infrastructure barrier to unified vulnerability detection.

7.2 Research Direction

CodeBERT’s demonstrated multi-language capabilities position transformer-based approaches as promising candidates for unified detection. However, addressing the identified gaps requires coordinated progress:

Dataset Development: Creating comprehensive multi-language vulnerability datasets with consistent labeling across languages, including examples that span language boundaries, represents the foundational infrastructure need.

Leveraging Pre-trained Models: Building on CodeBERT’s multilingual pre-training to validate whether vulnerability patterns learned in data-rich languages transfer to languages with sparse datasets.

Hybrid Architectures: Combining transformer semantic understanding with structural reasoning through architectures that integrate explicit control-flow and data-flow information. Such hybrid approaches remain largely unexplored, representing a significant research opportunity.

Extended Context: Developing mechanisms to extend analysis beyond individual functions to capture inter-procedural vulnerabilities common in polyglot systems.

Boundary Modeling: Explicitly representing and analyzing cross-language boundaries where memory management conflicts, type conversion issues, and security mechanism inconsistencies occur.

The convergence of demonstrated multi-language capabilities in CodeBERT, cross-language vulnerability pattern consistency, and the practical need for unified detection tools creates a compelling foundation for this research direction. Moving beyond the current fragmented landscape toward unified approaches would address both practical deployment challenges and fundamental security risks emerging at language interfaces in polyglot systems.

References

- [1] S. Cao, X. Sun, L. Bo, Y. Wei, and B. Li, “BGNN4VD: Constructing bidirectional graph neural-network for vulnerability detection,” *Inf. Softw. Technol.*, vol. 136, p. 106576, 2021.
- [2] X. Chen *et al.*, “Challenges and contributing factors in the utilization of large language models (LLMs),” arXiv preprint arXiv:2310.13343, 2023.
- [3] Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. Wagner, “DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection,” arXiv preprint arXiv:2304.00409, 2023.
- [4] D. Cotroneo, R. De Luca, and P. Liguori, “DeVAIC: A tool for security assessment of AI-generated code,” *Inf. Softw. Technol.*, vol. 177, p. 107572, 2025.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. 2019 Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol.*, vol. 1, 2019, pp. 4171–4186.
- [6] K. Ethayarajh, “How contextual are contextualized word representations? Comparing the geometry of BERT, ELMo, and GPT-2 embeddings,” in *Proc. 2019 Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, Hong Kong, China, 2019, pp. 55–65, doi: 10.18653/v1/D19-1006.
- [7] A. Ettinger, “What BERT is not: Lessons from a new suite of psycholinguistic diagnostics for language models,” *Trans. Assoc. Comput. Linguistics*, vol. 8, pp. 34–48, 2020, doi: 10.1162/tacl.a-00298.
- [8] Z. Feng *et al.*, “CodeBERT: A pre-trained model for programming and natural languages,” arXiv preprint arXiv:2002.08155, 2020.
- [9] D. Guo *et al.*, “UniXcoder: Unified cross-modal pre-training for code representation,” in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics*, 2022, pp. 7212–7225.
- [10] D. Guo *et al.*, “GraphCodeBERT: Pre-training code representations with data flow,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2021.
- [11] J. Hewitt and C. D. Manning, “A structural probe for finding syntax in word representations,” in *Proc. 2019 Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol.*, vol. 1, 2019, pp. 4129–4138.
- [12] D. Hin, A. Kan, H. Chen, and M. A. Babar, “LineVD: Statement-level vulnerability detection using graph neural networks,” in *Proc. 19th Int. Conf. Mining Softw. Repositories*, 2022, pp. 596–607.
- [13] P. Mayer, M. Kirsch, and M. A. Le, “On multi-language software development, cross-language links and accompanying tools: A survey of professional software developers,” *J. Softw. Eng. Res. Develop.*, vol. 5, no. 1, p. 1, 2017.
- [14] P. Michel, O. Levy, and G. Neubig, “Are sixteen heads really better than one?,” in *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.

- [15] G. Mussbacher *et al.*, “Polyglot software development: Wait, what?,” *IEEE Softw.*, vol. 41, no. 4, pp. 124–133, 2024.
- [16] A. Rogers, O. Kovaleva, and A. Rumshisky, “A primer in BERTology: What we know about how BERT works,” *Trans. Assoc. Comput. Linguistics*, vol. 8, pp. 842–866, 2020, doi: 10.1162/tacl.a.00349.
- [17] S. Sakharkar, “Systematic review: Analysis of coding vulnerabilities across languages,” *J. Inf. Security*, vol. 14, no. 4, pp. 330–342, 2023.
- [18] R. Syed, “Cybersecurity vulnerability management: A conceptual ontology and cyber intelligence alert system,” *Inf. Manage.*, vol. 57, no. 6, p. 103334, 2020.
- [19] I. Tenney, D. Das, and E. Pavlick, “BERT rediscovers the classical NLP pipeline,” in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, 2019, pp. 4593–4601, doi: 10.18653/v1/P19-1452.
- [20] A. Vaswani *et al.*, “Attention is all you need,” in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [21] E. Voita, D. Talbot, F. Moiseev, R. Sennrich, and I. Titov, “Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned,” in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, 2019, pp. 5797–5808, doi: 10.18653/v1/P19-1580.
- [22] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, “CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2021, pp. 8696–8708.
- [23] R. Xu *et al.*, “Detecting code vulnerabilities by learning from large-scale open source repositories,” *J. Inf. Security Appl.*, vol. 69, p. 103293, 2022.
- [24] H. Yang, W. Lian, S. Wang, and H. Cai, “Demystifying issues, challenges, and solutions for multilingual software development,” in *Proc. 2023 IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 1840–1852.
- [25] J. Zhou *et al.*, “Graph neural networks: A review of methods and applications,” *AI Open*, vol. 1, pp. 57–81, 2020.
- [26] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, “Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks,” in *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.

⁰The authors acknowledge the use of AI language models (Claude, Anthropic) for literature synthesis, text editing, and generation of illustrative code examples demonstrating cross-language vulnerability patterns (Section 3). All content and examples were verified against original sources and established vulnerability documentation.